

Sistemas Operativos

1º Parcial 2C2024 – TM – Resolución

Aclaración: La mayoría de las preguntas o ejercicios no tienen una única solución. Por lo tanto, si una solución particular no es similar a la expuesta aquí, no significa necesariamente que la misma sea incorrecta. Ante cualquier duda, consultar con el/la docente del curso.

Teoría

1. La CPU lanzaría una interrupción (sincrónica) al intentar ejecutar la instrucción, al atenderse esta interrupción se realizaría un cambio de modo y contexto para que el SO atienda la misma y usualmente finaliza al proceso que produjo la interrupción.
2. Aunque hay muchos criterios para analizar, podemos dar 2 ejemplos:
Si la prioridad del sistema fuera la **seguridad** y se deseara que una parte no pueda acceder a los datos de otra, utilizaría múltiples procesos cooperativos, ya que los hilos por defecto comparten memoria con sus hilos pares.
Si en cambio fuera una mayor prioridad la **eficiencia** en el uso de la memoria, utilizaría múltiples hilos dentro de un mismo proceso para que puedan reutilizar memoria lo más posible.
3. a. Suponiendo que ya se conoce la prioridad de cada cola y su quantum, se necesitaría saber además:
 - A qué cola/s ingresan los procesos nuevos?
 - La prioridad entre colas es con desalojo?
 - Si son colas retroalimentadas, bajo qué eventos se moverían procesos de una cola a otra?b. Depende del criterio en que los procesos se muevan de una cola a otra (cambian de prioridad), pero normalmente este algoritmo provocaría starvation en la cola menos prioritaria si siempre hay para ejecutar procesos en las más prioritarias.
4. a. Al implementarse con espera activa, los semáforos no toman valores negativos y seguirá ejecutando aquél que sea elegido por el planificador de corto plazo.
b. En todo caso ocurriría un **livelock**, ya que los procesos esperando por estos semáforos no estarían bloqueados sino en ejecución.
5. Un mecanismo sencillo pero que puede llegar a ser muy poco eficiente es que estos procesos pidan preventivamente todos los recursos que utilizarán a lo largo de su ejecución.

1.



Fin de I/O: t9, t16, t22

```
clientesEnFila = 0
```

```
preguntaColor = 0
```

```
respuestaColor = 0
```

```
posnet = 0
```

pagoRealizado = 0

entregarlphone = 0

mutexStock = 1

```
contadorIphones = [0, 0, 0]
```

Cliente (N instancias)	Vendedor tienda (1 instancia)	Camión distribuidor (1 instancia)
<pre> ponerseEnLaFila(); signal(clientesEnFila) wait(preguntaColor) responderColor(); signal(respuestaColor) wait(posnet) pagar(); signal(pagoRealizado) wait(entregarlphone) retirarse(); </pre>	<pre> while (1) { wait(clientesEnFila) preguntarColor(); signal(preguntaColor) wait(respuestaColor) int id_color = obtenerColorElegido(); wait(contadorlphones[id_color]) ingresarMontoAlPosnet(); signal(posnet) wait(pagoRealizado) wait(mutexStock) actualizarStock(STOCK); signal(mutexStock) signal(entregarlphone) } </pre>	<pre> while (1) { irATienda(); signal(contadorlphones[0]) x100 signal(contadorlphones[1]) x100 signal(contadorlphones[2]) x100 wait(mutexStock) actualizarStock(STOCK); signal(mutexStock) irADeposito(); } </pre>

3.

- a. No podría generarse deadlock entre los procesos 2 y 3 ya que piden los recursos en un mismo orden, por lo tanto, nunca se generaría espera circular que es condición necesaria para la ocurrencia de Deadlock.
- b. El SO podría estar implementando la estrategia de evasión, particularmente si el recurso A ya estuviera tomado, sería prudente bloquear al proceso 1 al pedir el recurso B ya que su asignación podría causar deadlock si luego éste mismo pidiera el recurso A.